BÖLÜM

3

Veri Tipleri, Tür Dönüşümü ve Kontrol Yapıları



BU BÖLÜMÜN SONUNDA

- C#'taki temel veri tiplerini, boyutlarını ve kullanım yerlerini bilecek,
- Formdan alınan string verileri sayıya, sayıları string'e dönüştürebilecek,
- Bölge ayarı, tamsayı bölmesi ve metin birleştirme tuzaklarını tanıyacak,
- Atama, karşılaştırma ve mantıksal operatörleri kullanabilecek,
- if–else, for ve while yapılarını form uygulamalarında kullanabilecek,
- Math sınıfının metotlarıyla hesaplama yapabileceksiniz.

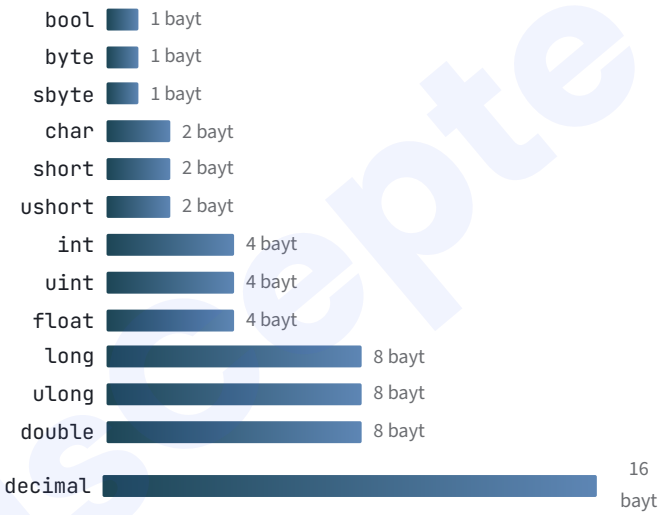
KampüsCepte · öğrenciler için ücretsiz ders notları

Form ekranında görünen her şey metindir. Oysa programlar sayılarla hesap yapar, karar verir ve işleri tekrarlar. Bu bölümde formdan veri almayı, onu doğru türe çevirmeyi ve karar ile döngü yapılarıyla işlemeyi öğreniyoruz.

3.1 Veri Tipleri

Bir değişken tanımlanırken bellekte ne kadar yer kaplayacağı ve hangi değerleri alabileceği **veri tipi** ile belirlenir. C#'ta temel tipler üç grupta toplanır: **sayısal tipler** (byte, sbyte, short, ushort, int, uint, long, ulong, float, double, decimal), **metinsel tipler** (char, string) ve **mantıksal tip** (bool).

Tip	Boyut	Değer aralığı / açıklama
`byte`	1 bayt	İşaretsiz tamsayı: 0 … 255
`sbyte`	1 bayt	İşaretli tamsayı: −128 … 127
`short`	2 bayt	İşaretli tamsayı: −32 768 … 32 767
`ushort`	2 bayt	İşaretsiz tamsayı: 0 … 65 535
`int`	4 bayt	İşaretli tamsayı: yaklaşık ±2,1 milyar
`uint`	4 bayt	İşaretsiz tamsayı: 0 … yaklaşık 4,3 milyar
`long`	8 bayt	İşaretli tamsayı: yaklaşık $\pm 9{,}2 \times 10^{18}$
`ulong`	8 bayt	İşaretsiz tamsayı: 0 … yaklaşık $1{,}8 \times 10^{19}$
`float`	4 bayt	Ondalıklı sayı, yaklaşık 7 basamak duyarlık
`double`	8 bayt	Ondalıklı sayı, yaklaşık 15–16 basamak duyarlık
`decimal`	16 bayt	Ondalıklı sayı, 28–29 basamak; para hesapları için
`char`	2 bayt	Tek bir Unicode karakter: `'A'`
`string`	değişken	Karakter dizisi (metin): `"Merhaba"`
`bool`	1 bayt	Yalnızca `true` veya `false`



Şekil 3.1 *Tiplerin bellekte kapladığı alan*

KampüsCepte · öğrenciler için ücretsiz ders notları

DİKKAT

Bazı kaynaklarda `bool` tipinin 2 bayt olduğu yazar. C#'ta `sizeof(bool)` değeri **1**'dir. Benzer biçimde string için "2 milyar karaktere kadar" ifadesi geçer. .NET'te bir string'in alabileceği en fazla uzunluk yaklaşık **1 milyar** (1 073 741 791) karakterdir ve pratikte sınırı bilgisayarın belleği belirler.

DEĞİŞKEN TANIMLAMA ÖRNEKLERİ

```
int ogrenciSayisi = 45;
double ortalama = 72.5;
float oran = 0.75f;        // float sabitleri f son eki alır
decimal fiyat = 149.90m;   // decimal sabitleri m son eki alır
char harf = 'A';           // tek tırnak, tek karakter
string ad = "Yunus";       // çift tırnak, metin
bool aktif = true;         // yalnızca true veya false
```

Ondalıklı bir sabit (ör. `72.5`) C#'ta varsayılan olarak **double** kabul edilir. float'a atamak için sonuna `f`, decimal'e atamak için `m` eklenir. Bu son ek unutulursa derleyici "double türü kendiliğinden float/decimal'e dönüştürülemez" hatası verir. Kod içindeki ondalık ayırıcı, bilgisayarın dil ayarı ne olursa olsun her zaman **nokta**dır.

3.2 Formdan Veri Almak: Tür Dönüşümleri

Form üzerinden aldığımız veriler (TextBox, Label, ComboBox'ın `Text` özelliği) **string** türündedir. Girilen veri sayı olarak kullanılacaksa önce sayıya **çevrilmelidir**. Tersine, forma yazdıracağımız bir sayıyı da önce string'e çevirmemiz gerekir.



Tür dönüşümü: veriyi kullanılacağı yerin biçimine uyarlamak

KampüsCepte · öğrenciler için ücretsiz ders notları

TEMEL DÖNÜŞÜMLER

```
// string → sayı
int yas = Convert.ToInt32(textBox1.Text);
double boy = Convert.ToDouble(textBox2.Text);
int adet = int.Parse(textBox3.Text);

// sayı → string
label1.Text = yas.ToString();          // 1. yol: ToString()
label2.Text = Convert.ToString(boy);   // 2. yol: Convert.ToString()
```

Yöntem	Ne yapar?	Geçersiz giriş ("abc") olursa
`Convert.ToInt32(metin)`	Metni int'e çevirir	FormatException hatası
`Convert.ToDouble(metin)`	Metni double'a çevirir	FormatException hatası
`int.Parse(metin)`	Metni int'e çevirir	FormatException hatası
`double.TryParse(metin, out x)`	Çevirmeyi **dener**; başarılıysa `true` döner ve sonucu x'e yazar	Hata vermez, `false` döner
`sayi.ToString()`	Sayıyı metne çevirir	—
`Convert.ToString(sayi)`	Sayıyı metne çevirir	—

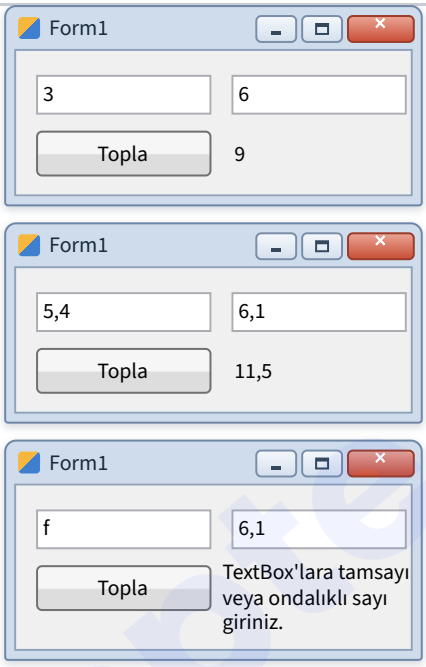
Kullanıcıdan gelen veriye hiçbir zaman güvenilmemelidir. Boş bırakılan bir kutu ya da yanlışlıkla yazılan bir harf, `Convert` kullanan programı hata vererek durdurur. Bu yüzden kullanıcı girişlerinde `TryParse` tercih edilir. Aşağıda, iki sayıyı toplayan programın hem ders sunumundaki hâli hem de hatalı girişe dayanıklı hâli verilmiştir.

KampüsCepte

KampüsCepte · öğrenciler için ücretsiz ders notları

SUNUMDAKİ ÇÖZÜM

```
private void button1_Click(object sender, EventArgs
e)
{
    double sayi1, sayi2, sonuc;
    sayi1 = Convert.ToDouble(textBox1.Text);
    sayi2 = Convert.ToDouble(textBox2.Text);
    sonuc = sayi1 + sayi2;
    label1.Text = Convert.ToString(sonuc);
}
```



DAYANIKLI ÇÖZÜM (TRYPARSE)

```
private void button1_Click(object sender, EventArgs
e)
{
    double sayi1, sayi2;
    if (double.TryParse(textBox1.Text, out sayi1) &&
double.TryParse(textBox2.Text, out sayi2))
    {
        label1.Text = (sayi1 + sayi2).ToString();
    }
    else
    {
        label1.Text = "TextBox'lara tamsayı veya
ondalıklı sayı giriniz.";
    }
}
```

DİKKAT

Bölge ayarı tuzağı. `Convert.ToDouble` ve `double.Parse`, bilgisayarın **bölge ayarına** göre çalışır. Türkçe ayarlı bir bilgisayarda ondalık ayırıcı **virgül**, binlik ayırıcı **nokta**dır. Bu yüzden:

Kutuya yazılan	Türkçe Windows'ta Convert.ToDouble sonucu
`5,4`	5,4
`5.4`	**54** (nokta binlik ayırıcı sayılır)

Sunumdaki ekran görüntülerinde sayıların 23,44 gibi virgülle yazılmasının nedeni budur. Kod içinde ise ondalık ayırıcı her zaman noktadır: `double x = 5.4;`

DİKKAT

Birleştirme tuzağı. `+` işleci iki sayı arasında toplama, bir metin ile herhangi bir değer arasında **birleştirme** yapar. `textBox1.Text + textBox2.Text` ifadesi, kutularda 3 ve 6 yazıyorsa 9 değil **"36"** sonucunu verir. Toplama yapabilmek için önce dönüştürmek şarttır.

KampüsCepte · öğrenciler için ücretsiz ders notları

3.3 Operatörler

Operatör	Eşdeğeri	Anlamı
`x += y`	`x = x + y`	x + y'nin değerini x'e ata
`x -= y`	`x = x - y`	x − y'nin değerini x'e ata
`x *= y`	`x = x * y`	x · y'nin değerini x'e ata
`x /= y`	`x = x / y`	x / y'nin değerini x'e ata
`x %= y`	`x = x % y`	x'in y'ye bölümünden kalanı x'e ata
`x++`	`x = x + 1`	x'i bir artır
`x--`	`x = x - 1`	x'i bir azalt

Operatör	Anlamı
`==`	Eşittir
`!=`	Eşit değil
`>` `<`	Büyük, küçük
`>=` `<=`	Büyük eşit, küçük eşit
`!`	Değil
`&&`	Ve (iki şart da doğru)
`||`	Veya (en az biri doğru)

DİKKAT

Tamsayı bölmesi. İki **int** değer bölündüğünde sonuç da int olur ve ondalık kısım **atılır** (yuvarlanmaz): `7 / 2` → **3**, `10 / 4` → **2**. Sonuç bir double değişkene atansa bile bölme önce int olarak yapılır: `double s = 7 / 2;` ifadesinde s'nin değeri 3'tür. Doğru sonuç için işlenenlerden en az biri ondalıklı olmalıdır: `7 / 2.0` → 3,5. Kalan operatörü `%` ise bölümden kalanı verir: `10 % 4` → 2.

Atama için tek eşittir (`=`), karşılaştırma için çift eşittir (`==`) kullanılır. `if (a = 3)` yazımı derleme hatası verir.

3.4 if – else Yapısı

Program, bir şartın doğru ya da yanlış olmasına göre farklı komutlar çalıştırır. Şart doğruysa `if` bloğu, yanlışsa `else` bloğu çalışır. `else` bloğu isteğe bağlıdır; yazılmazsa şart yanlış olduğunda hiçbir şey yapılmaz. Birden çok şart sırayla denenecekse `else if` kullanılır. Bu yapıda doğru bulunan **ilk** şartın bloğu çalışır, kalanlara bakılmaz.



if – else: şarta göre iki yoldan biri seçilir

KampüsCepte · öğrenciler için ücretsiz ders notları

TEXTBOX'A GİRİLEN SAYIYA GÖRE MESAJ

```
private void button1_Click(object sender, EventArgs e)
{
    int a = Convert.ToInt32(textBox1.Text);

    if (a < 0)
    {
        label1.Text = "TextBox'a pozitif bir sayı giriniz.";
    }
    else if (a == 3)
    {
        label1.Text = "TextBox'a 3 sayısını girdiniz.";
    }
    else
    {
        label1.Text = "";
    }
}
```

İPUCU

Blokta tek bir komut varsa süslü parantez yazılmayabilir: `if (a < 0) label1.Text = "…";` . Ancak sonradan ikinci bir komut eklendiğinde parantez unutulursa ikinci komut şarttan bağımsız çalışır. Bu yüzden parantez kullanmak iyi bir alışkanlıktır.

3.5 Döngüler: for ve while

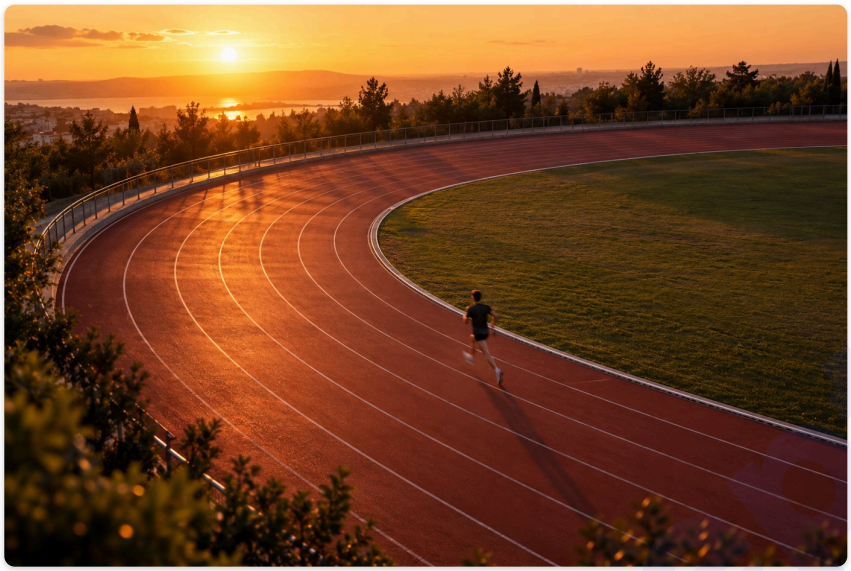
GENEL YAPI

```
for (başlangıç işlemleri; devam şartı; ara işlemler)
{
    komutlar;
}

while (şart)
{
    komutlar;
}
```


KampüsCepte · öğrenciler için ücretsiz ders notları

for döngüsü, devam şartı sağlandığı sürece komutları tekrarlar. Başlangıç işlemleri yalnızca bir kez, en başta çalışır. Devam şartı her turdan önce kontrol edilir ve **bool** türündedir. Ara işlemler (genellikle sayacın artırılması) her turun sonunda çalışır.



Döngü: şart sağlandıkça aynı tur tekrar tekrar koşulur

FOR: FORMUN BAŞLIĞINDA SAYMA

```
private void button1_Click(object sender, EventArgs e)
{
    int N = 20;
    for (int i = 0; i < N; i++)
    {
        this.Text = i.ToString();   // formun başlık çubuğuna yazar
    }
}
```

Burada int tipinde N değişkenine 20 değeri atanmıştır. Döngüde int tipinde `i` tanımlanır ve başlangıç değeri 0 olur. `i` , N ile karşılaştırılır. i N'den küçük olduğu sürece döngü çalışır ve her turun sonunda `i++` ile bir artar. Her turda i'nin değeri formun başlık çubuğuna (`this.Text`) yazılır. Değer 0'dan 19'a kadar değişir, ancak döngü bir anda bittiği için kullanıcı pratikte yalnızca son değeri, **19**'u görür. Döngü bittiğinde i = 20 olmuştur ve şart (20 < 20) yanlıştır.

KampüsCepte · öğrenciler için ücretsiz ders notları

while döngüsü, şart doğru olduğu sürece çalışır. Sayacın artırılması bloğun içinde yapılmalıdır, aksi hâlde şart hep doğru kalır ve döngü sonsuza kadar sürer. Form uygulamasında sonsuz döngü, programın donmasına (yanıt vermemesine) yol açar.

WHİLE: 1'DEN A'YA KADAR TOPLAM

```
private void button1_Click(object sender, EventArgs e)
{
    int a = Convert.ToInt32(textBox1.Text);   // üst sınır, örneğin 5
    int b = 0;                                // toplam
    int c = 1;                                // sayaç

    while (c <= a)
    {
        b += c;
        c++;
    }
    label1.Text = "1 ile " + a + " arasındaki sayıların toplamı: " + b;
}
```

Tur	c (başta)	c <= 5 ?	b += c	c++
1	1	doğru	1	2
2	2	doğru	3	3
3	3	doğru	6	4
4	4	doğru	10	5
5	5	doğru	15	6
—	6	**yanlış** → döngü biter	15	—

Tabloda kutuya 5 girildiğinde döngünün her turu izlenmiştir. Bu tür tabloya **izleme tablosu** denir ve bir döngünün ne yaptığını anlamanın en güvenilir yoludur.

3.6 Math Sınıfı

Matematiksel işlemler için **Math** sınıfının hazır metotları ve sabitleri kullanılır. Hepsi **statik**tir. Kullanmak için nesne oluşturulmaz, doğrudan sınıf adıyla çağrılır: `Math.Sqrt(16)`.

Üye	Görevi	Örnek → sonuç
`Math.E`	e sayısı	≈ 2,71828
`Math.PI`	π sayısı	≈ 3,14159
`Math.Sin(b)` `Cos` `Tan`	Trigonometrik fonksiyonlar; b **radyan** cinsindendir	Dereceyi radyana çevirmek için π/180 ile çarpılır
`Math.Exp(b)`	e^b	Math.Exp(1) → 2,718…
`Math.Pow(b, c)`	b^c	Math.Pow(2, 3) → 8
`Math.Sqrt(b)`	Karekök	Math.Sqrt(16) → 4
`Math.Ceiling(b)`	Yukarı (büyük olan tamsayıya) yuvarlar	10,3 → 11 · −10,3 → **−10**

KampüsCepte · öğrenciler için ücretsiz ders notları

Üye	Görevi	Örnek → sonuç
`Math.Floor(b)`	Aşağı (küçük olan tamsayıya) yuvarlar	10,3 → 10 · −10,3 → **−11**
`Math.Round(b)`	En yakın tamsayıya yuvarlar	10,3 → 10 · 10,8 → 11
`Math.Round(b, c)`	Virgülden sonra c basamak kalacak şekilde yuvarlar	Math.Round(10.3867, 1) → 10,4
`Math.Min(b, c)` `Math.Max(b, c)`	Küçük / büyük olanı verir	Min(3, 4) → 3 · Max(3, 4) → 4
`Math.Abs(b)`	Mutlak değer	Math.Abs(−3) → 3
`Math.Log10(b)`	10 tabanında logaritma	Math.Log10(100) → 2
`Math.Log(b)`	Doğal logaritma (ln)	Math.Log(Math.E) → 1
`Math.Log(b, c)`	c tabanında logaritma	Math.Log(8, 2) → 3

DİKKAT

Yarım değerlerde yuvarlama. `Math.Round` tam ortadaki (,5) değerleri varsayılan olarak **çift sayıya** yuvarlar (bankacı yuvarlaması): `Math.Round(2.5)` → **2**, `Math.Round(3.5)` → **4**. Okulda öğrendiğimiz "5 ve üstü yukarı" kuralı isteniyorsa ikinci parametre verilir: `Math.Round(2.5, MidpointRounding.AwayFromZero)` → 3.

3.7 Form_Load ile Başlangıç Değerleri

Form1_Load olayına yazılan kodlar program yüklenirken çalışır (*load*: yüklemek). Aşağıdaki ikinci derece denklem çözücüsünde kutulara açılışta örnek değerler yazılmıştır: a = 1, b = 0, c = −4. Bu denklemin ($x^2 - 4 = 0$) kökleri 2 ve −2'dir. Hesaplanan sonuç forma yazdırılırken sayı iki farklı yolla string'e çevrilmiştir: **1. yol** `Convert.ToString(...)`, **2. yol** `.ToString()`.

İKİNCİ DERECE DENKLEM: AX² + BX + C = 0

```
private void Form1_Load(object sender, EventArgs e)
{
    textBox1.Text = "1";
    textBox2.Text = "0";
    textBox3.Text = "-4";
    textBox4.Text = "birinci kök";
    textBox5.Text = "ikinci kök";
}

private void button1_Click(object sender, EventArgs e)
{
    double a, b, c, delta, x1, x2;
    a = Convert.ToDouble(textBox1.Text);   // textBox1'in metni double'a çevrilip a'ya atanır
    b = Convert.ToDouble(textBox2.Text);
    c = Convert.ToDouble(textBox3.Text);

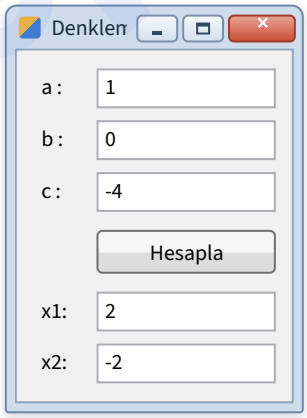
    if (a == 0)
    {
        MessageBox.Show("a katsayısı 0 olamaz.");
        return;
    }
```


KampüsCepte · öğrenciler için ücretsiz ders notları

```
    delta = b * b - 4 * a * c;

    if (delta > 0)
    {
        x1 = (-b + Math.Sqrt(delta)) / (2 * a);
        x2 = (-b - Math.Sqrt(delta)) / (2 * a);
        textBox4.Text = x1.ToString();                    // 2. yol
        textBox5.Text = x2.ToString();
    }
    else if (delta == 0)
    {
        textBox4.Text = Convert.ToString(-b / (2 * a));   // 1. yol
        textBox5.Text = textBox4.Text;                    // çakışık (eşit) kök
    }
    else
    {
        textBox4.Text = "Reel kök yok";
        textBox5.Text = "Reel kök yok";
    }
}
```

Form açıldığında Load olayı kutulara 1, 0 ve −4 değerlerini yazar. **Hesapla** butonuna basıldığında delta = 0² − 4·1·(−4) = 16 bulunur. Delta pozitif olduğu için iki farklı kök hesaplanır: x1 = (0 + 4) / 2 = **2**, x2 = (0 − 4) / 2 = **−2**.



Sunumdaki örnekte delta kontrol edilmeden kökler hesaplanmıştı. Delta negatif olduğunda `Math.Sqrt` hata vermez, sonuç olarak **NaN** (Not a Number: sayı değil) üretir ve kutulara anlamsız bir değer yazılır. a = 0 olduğunda ise sıfıra bölme oluşur. Yukarıdaki kodda bu iki durum da kontrol edilmiştir. Delta sıfır olduğunda iki kök eşittir ve tek formülle (−b / 2a) hesaplanır.

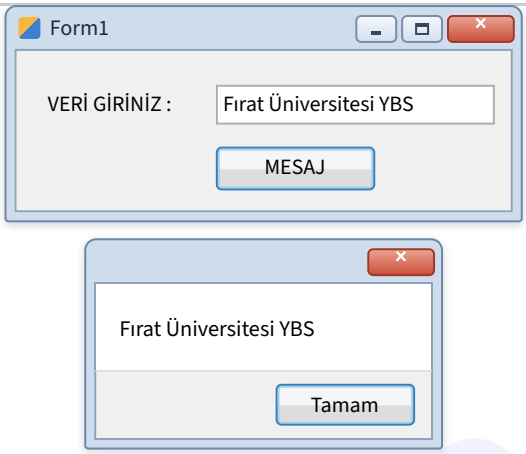
3.8 Çözümlü Uygulamalar

Uygulama 1 — Kutudaki veriyi mesaj olarak gösterme

VERİ GİRİNİZ yazan bir Label, bir TextBox ve MESAJ yazan bir buton forma eklenir. Butona tıklanınca kutuya yazılan veri mesaj olarak gösterilir.

KampüsCepte · öğrenciler için ücretsiz ders notları

```
private void button1_Click(object sender,
EventArgs e)
{
    MessageBox.Show(textBox1.Text);
}
```

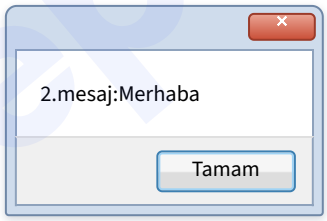


Uygulama 2 — Mesajı 10 kez gösterme

Aynı form kullanılarak kutudaki veri 10 kez, başında sıra numarasıyla gösterilir. `i + ".mesaj:"` ifadesinde int ile string toplandığı için sayı metne çevrilip birleştirilir.

HER MESAJ AYRI PENCEREDE

```
private void button1_Click(object sender, EventArgs e)
{
    for (int i = 1; i <= 10; i++)
    {
        MessageBox.Show(i + ".mesaj:" + textBox1.Text);
    }
}
```



Bu çözümde kullanıcı on kez **Tamam**'a basmak zorundadır. Mesajları tek bir string'de biriktirip tek seferde göstermek daha kullanışlıdır. `\n` yeni satır karakteridir:

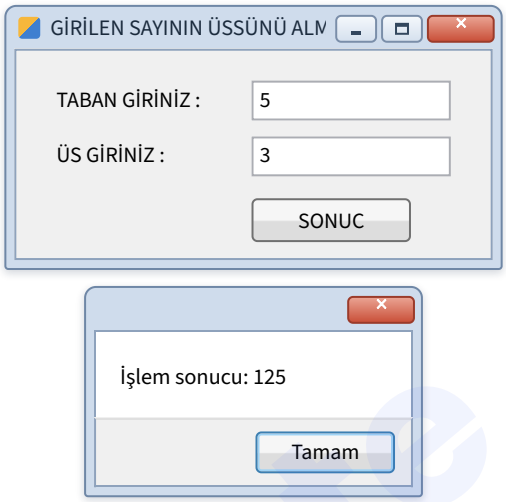
TÜM MESAJLAR TEK PENCEREDE

```
private void button1_Click(object sender, EventArgs e)
{
    string mesaj = "";
    for (int i = 1; i <= 10; i++)
    {
        mesaj += i + ". mesaj: " + textBox1.Text + "\n";
    }
    MessageBox.Show(mesaj);      // 10 satır tek pencerede
}
```


KampüsCepte · öğrenciler için ücretsiz ders notları

Uygulama 3 — Üs alma (Math.Pow)

```
private void button1_Click(object sender, EventArgs
e)
{
    double taban, us;
    taban = Convert.ToDouble(textBox1.Text);   //
metin ondalıklı sayıya çevrilir
    us = Convert.ToDouble(textBox2.Text);
    double sonuc = Math.Pow(taban, us);
    MessageBox.Show("İşlem sonucu: " + sonuc);
}
```



Uygulama 4 — Dört işlem (RadioButton)

İki sayı girilir, **İŞLEM SEÇ** bölümünden hangi radyo buton seçilmişse **İŞLEM SONUCU** butonuna tıklandığında o işlemin sonucu gösterilir. Sunumdaki çözüm aşağıdadır. 2 ve 4 sayıları için çarpma seçilirse 8, bölme seçilirse 0,5 sonucu görünür.

SUNUMDAKİ ÇÖZÜM

```
private void button1_Click(object sender, EventArgs e)
{
    double sayi1, sayi2, sonuc;

    sayi1 = Convert.ToDouble(textBox1.Text);
    sayi2 = Convert.ToDouble(textBox2.Text);
    if (radioButton1.Checked == true) { sonuc = sayi1 + sayi2; MessageBox.Show("" + sonuc); }
    if (radioButton2.Checked == true) { sonuc = sayi1 - sayi2; MessageBox.Show("" + sonuc); }
    if (radioButton3.Checked == true) { sonuc = sayi1 * sayi2; MessageBox.Show("" + sonuc); }
    if (radioButton4.Checked == true) { sonuc = sayi1 / sayi2; MessageBox.Show("" + sonuc); }
}
```



Şekil 3.2 *Çarpma seçiliyken 2 ve 4 için sonuç*

`radioButton1.Checked == true` karşılaştırması gereksizdir. `Checked` zaten bool değerdir ve `if (radioButton1.Checked)` yazmak aynı anlama gelir. Bu çözüm çalışır, ancak geliştirilmeye açık noktaları vardır: hiçbir işlem seçilmezse kullanıcıya bilgi verilmez, sıfıra bölme kontrol edilmez ve aynı MessageBox satırı dört kez tekrarlanır. Bu çözümü iyileştirmek bu bölümün Kendini Test Et kısmında Soru 3.4 olarak size bırakılmıştır.

KampüsCepte · öğrenciler için ücretsiz ders notları

ÖZET

- Tamsayılar için çoğunlukla `int`, ondalıklı sayılar için `double`, para için `decimal`, metin için `string`, doğru/yanlış için `bool` (1 bayt) kullanılır.
- Formdan gelen her veri **string**'dir. Sayıya `Convert.ToInt32/ToDouble`, `Parse` ya da güvenli biçimde `TryParse` ile çevrilir. Sayı string'e `ToString()` ya da `Convert.ToString()` ile çevrilir.
- Türkçe sistemde ondalık ayırıcı virgüldür. `"5.4"` metni 54 olarak okunur.
- `+` metinle birlikte kullanılınca birleştirir. İki int'in bölümü int'tir (`7/2 = 3`).
- `if – else if – else` zincirinde doğru olan ilk bloğun kodu çalışır. `for` sayaçlı tekrarlar, `while` şarta bağlı tekrarlar içindir.
- Math sınıfının metotları statiktir. `Math.Round` yarım değerleri çift sayıya yuvarlar. Negatif sayının karekökü **NaN**'dır.

KampüsCepte

KampüsCepte · öğrenciler için ücretsiz ders notları